Experimento 3: contratos de una herramienta

Examen Parcial CC0F4 - Tópicos de Ciencia de la Computación I

¿De qué trata el experimento?

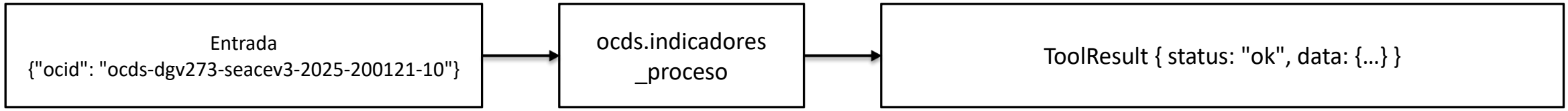
- Los experimentos 1 y 2 buscan texto y generan respuestas. Aquí no interviene ningún LLM
  - Se prueba una herramienta que el LLM podría llamar: ocds.indicadores_proceso
- La herramienta puede recibir argumentos mal escritos y puede fallar. El experimento mide cómo reacciona
- Pregunta del experimento
  - Con los mismos 16 casos, ¿qué agregan los contratos y el reintento selectivo frente a ejecutar sin contratos o reintentar todo?
- Se comparan cuatro políticas de ejecución, de A (sin nada) a D (contratos y reintento selectivo)
- La medida es objetiva: cada caso tiene un resultado esperado fijado antes de correr

La herramienta: ocds.indicadores_proceso

Señal	Regla	Para qué sirve
unico_postor	n_postores == 1	indicador de riesgo (Fazekas y Kocsis, 2020)
adjudicado_sobre_vr	monto adjudicado / valor referencial > 1	sobrecosto
equivalente_solo_en_objeto	el objeto dice "o equivalente" y el ítem no	la inconsistencia de Q15

- Recibe un ocid (y un ítem si el proceso tiene varios) y calcula las tres señales desde los datos OCDS
- No decide si un requisito está direccionado: eso sigue siendo del LLM y de la regla de la PC1
- Solo entrega hechos del registro, para contrastar lo que dice el modelo
    - Ejemplo con Q15: el LLM se contradice sobre "o equivalente" y la herramienta devuelve objeto sí, ítem no
- No tiene efectos: solo lee data/indice_procesos.jsonl

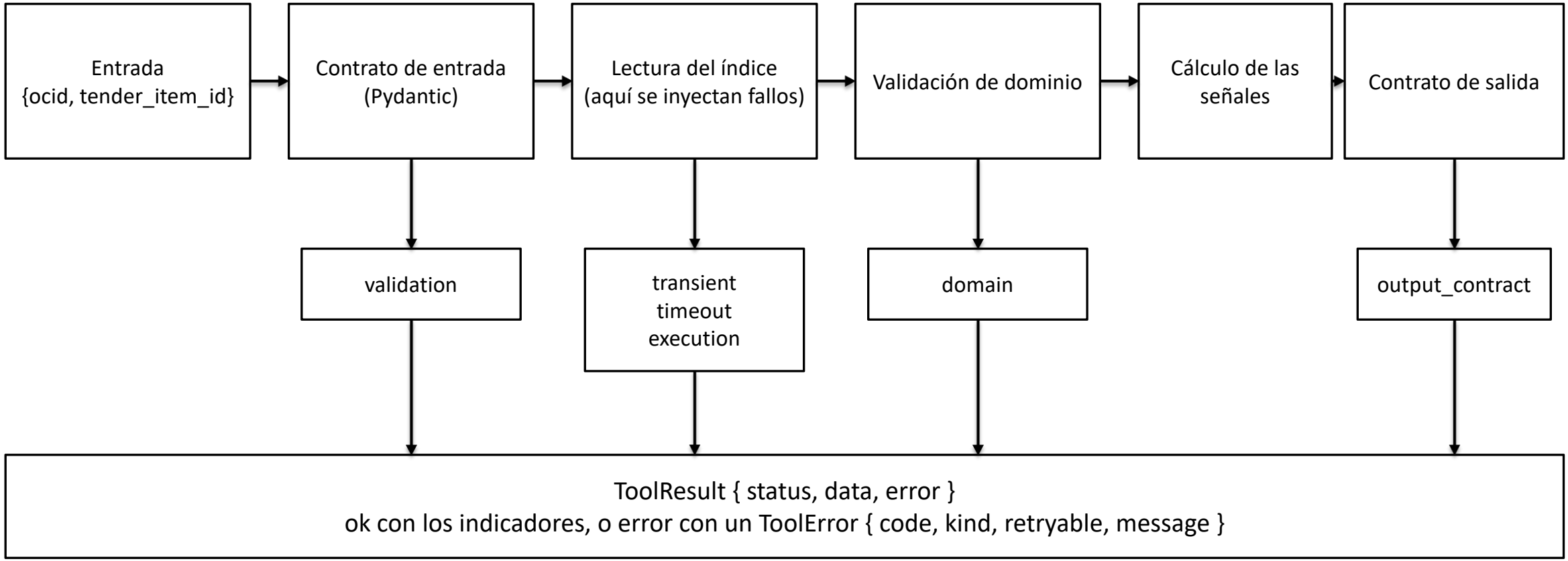
Ejemplo de una llamada válida (proceso de Q15)



Campo de data	Valor
nomenclatura	LP-ABR-1-2025-APN-1
entidad	MTC - AUTORIDAD PORTUARIA NACIONAL
n_postores / unico_postor	11 / false
ratio_adjudicado_vr / adjudicado_sobre_vr	0,7865 / false
objeto_dice_equivalente / item_dice_equivalente	true / false
alertas	["equivalente_solo_en_objeto"]

- El proceso no tuvo un solo postor ni se adjudicó sobre el valor referencial, pero sí tiene la alerta de Q15

Por dónde pasa una llamada



- El contrato de entrada y el de salida son los dos controles de la herramienta; los errores de abajo salen tipados
- Si el contrato de salida detecta una salida rota, no llega al LLM y no se reintenta

Los contratos

Parte	Qué exige
Entrada	ocid con el patrón ocds-dgv273-seacev3-N (hasta dos sufijos); tender_item_id entero estricto ≥ 1 u omitido
	extra="forbid": un argumento inventado es un error, no se ignora
Dominio	el ocid existe en el índice; el ítem pertenece al proceso; si hay varios ítems hay que indicar cuál
Salida	tipos y rangos (n_postores ≥ 0, ratio ≥ 0)
	coherencia: unico_postor coincide con n_postores, adjudicado_sobre_vr con el ratio, y alertas con los indicadores

- Un argumento mal escrito falla en la entrada y no dentro del cálculo
- Una salida incoherente se bloquea antes de que el LLM la lea

Los tipos de error

kind	Cuándo ocurre	¿Se reintenta?
validation	la entrada no cumple el contrato	no
domain	ocid o ítem que no existen, o falta elegir ítem	no
transient	el índice no está disponible por un momento	sí
timeout	un intento pasa de 100 ms	sí, si queda deadline
execution	excepción interna no esperada	no
output_contract	la salida no cumple el contrato	no
protocol	herramienta desconocida	no

- Cada error lleva retryable: el llamador sabe si tiene sentido volver a intentar

Cuándo reintentar y cuándo no

- La herramienta solo lee, así que repetir una llamada no duplica nada: reintentar es seguro
- Pero solo es útil si el error puede desaparecer solo
  - Sí: transient (el índice no responde un momento) y timeout (un intento pasa de 100 ms)
  - No: validation, domain, output_contract, execution y protocol
  - Reintentar una entrada inválida o un ocid inexistente da el mismo error otra vez
  - Una salida rota es un bug de la herramienta; repetirla da la misma salida rota
- Parámetros: hasta 3 intentos, espera de 20 y 40 ms entre intentos, deadline total de 300 ms
- Los fallos transitorios y lentos se inyectan de forma determinista, porque el índice es un archivo local y no falla solo

Las cuatro políticas

Política	Contratos y ToolError	Timeout	Reintento	Intentos máx.
A sin contratos	no	no	no	1
B contratos	sí	no	no	1
C reintenta todo	sí	100 ms por intento, deadline 300 ms	cualquier error	3
D reintento selectivo	sí	100 ms por intento, deadline 300 ms	solo transient y timeout	3

- A es la línea base: ejecuta lo que llegue, y los errores salen sin tipo
- B agrega los contratos y el ToolError tipado
- C agrega a B un timeout y reintenta cualquier error
- D agrega a B un timeout por intento y un deadline total, y reintenta solo lo que puede arreglarse solo

Los 16 casos

Grupo	Casos	Resultado que pide la especificación
Válidos (4)	alerta de Q15, único postor, sobre el VR, ítem de un proceso de 6 ítems	ok con la alerta esperada
Entrada inválida (3)	ocid mal formado, ítem como texto, argumento extra	validation
Error de dominio (3)	ocid inexistente, ítem de otro proceso, falta elegir ítem (104 ítems)	domain
Transitorio (2)	falla solo en el primer intento / falla siempre	ok tras reintentar / transient
Lento (2)	tarda 150 ms solo la primera vez / siempre	ok tras reintentar / timeout
Error interno (1)	una excepción dentro de la herramienta	execution
Salida mal formada (1)	devuelve n_postores = -1	output_contract

- El resultado esperado de cada caso está escrito en el código antes de correr (CASOS en exp3_herramienta.py)

Qué se mantiene igual

- Los mismos 16 casos, con los mismos argumentos y los mismos fallos inyectados, para las cuatro políticas
- El resultado esperado de cada caso, fijado antes de correr
- El índice de procesos, con su SHA-256 guardado en results/exp3/manifiesto.json
- Los tiempos: 3 intentos, esperas de 20 y 40 ms, 100 ms por intento y 300 ms de deadline
- No hay azar ni modelo: la misma entrada da siempre el mismo resultado
- Lo único que cambia entre políticas es lo que rodea a la herramienta, no la herramienta

Cómo se mide

- Casos que cumplen la especificación
  - Un caso cumple si el status y el tipo de error coinciden con lo esperado (y, si es ok, las alertas)
- Entradas inválidas ejecutadas: entradas que el contrato habría rechazado y que la política dejó correr
- Salidas inválidas aceptadas: salidas que rompen el contrato de salida y llegaron como ok
- Intentos totales
- Reintentos inútiles
  - Intentos hechos después de un error que reintentando no se arregla
- Latencia media y máxima por caso

Ejemplo: la misma entrada bajo A y bajo B-D

Caso	Política A (sin contratos)	Políticas B, C y D
Argumento extra {ocid, entidad: "HOSPITAL GENERAL DE JAEN"}	ok: ignora el argumento en silencio	validation: el argumento extra no existe
Ítem como texto tender_item_id: "21314522"	error genérico: busca el ítem como texto y no lo halla, como si no existiera	validation: el tipo del argumento es incorrecto
Salida mal formada n_postores = -1	ok con n_postores = -1	output_contract: se bloquea y no se reintenta

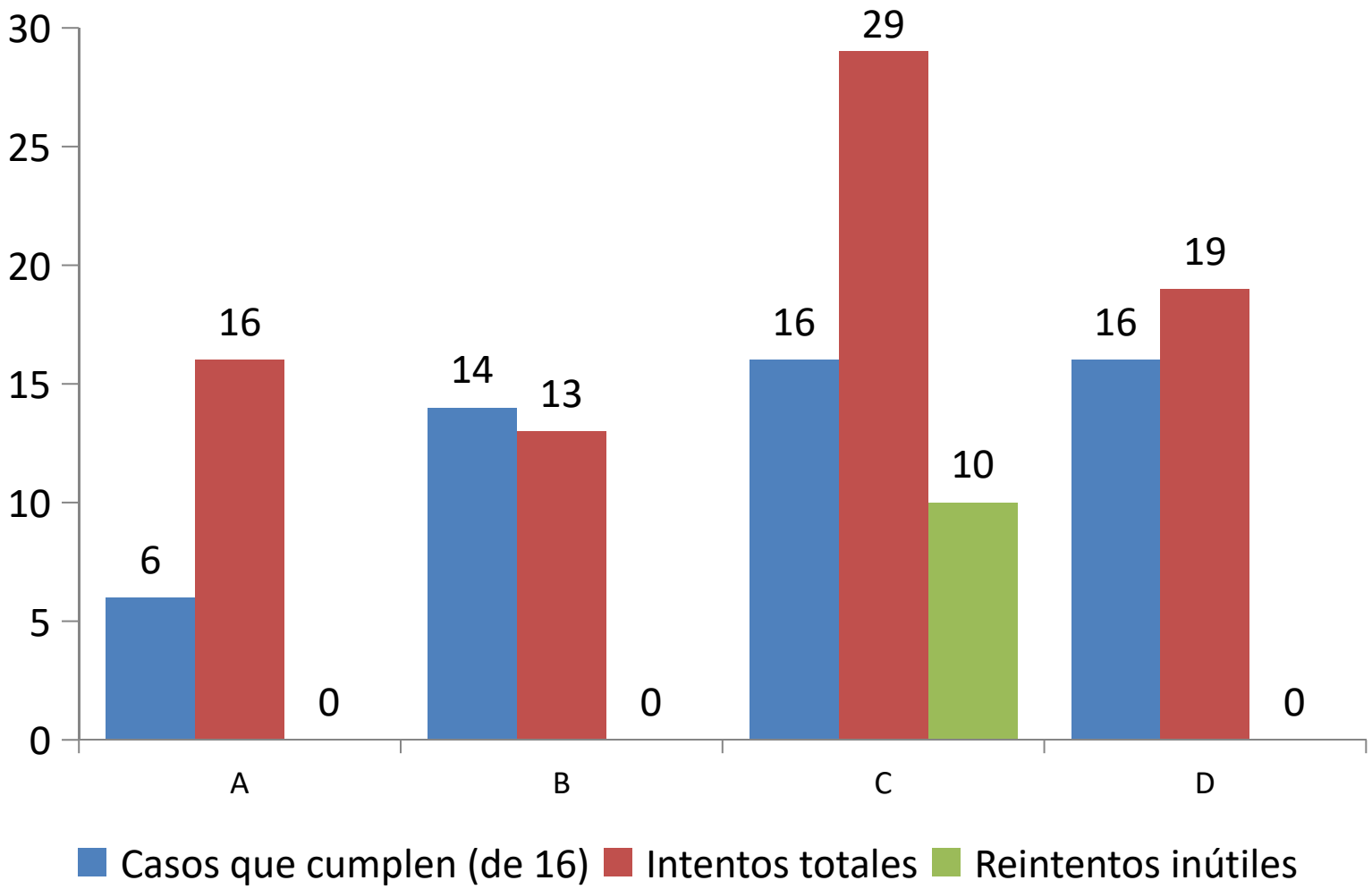
- Sin contrato, A acepta un dato falso y lo pasa al LLM como si fuera correcto
- El error de A en el ítem es engañoso: el ítem sí existe, lo que está mal es su tipo

Resultados

Política	Cumplen	Entradas inválidas ejecutadas	Salidas inválidas aceptadas	Intentos	Reintentos inútiles	Latencia media (ms)	Latencia máx. (ms)
A sin contratos	6/16	3	1	16	0	20	161
B contratos	14/16	0	0	13	0	20	160
C reintenta todo	16/16	0	0	29	10	58	301
D reintento selectivo	16/16	0	0	19	0	34	299

- C y D cumplen los 16 casos; B cumple 14 y A cumple 6
- D logra lo mismo que C con 19 intentos en vez de 29 y sin reintentos inútiles
- El detalle de cada caso está en cumplimiento_por_caso.csv y las trazas de cada intento en trazas.jsonl

Resultados: gráfico



- Azul: casos que cumplen; rojo: intentos; verde: reintentos inútiles
- Cumplir sube con cada capa: 6, 14, 16, 16
- C gasta 29 intentos, 10 de ellos inútiles
- D cumple lo mismo con 19

Resultados: los casos donde alguna política falla

Caso	A	B	C	D
invalido_ocid_mal_formado	no	sí	sí	sí
invalido_item_como_texto	no	sí	sí	sí
invalido_argumento_extra	no	sí	sí	sí
dominio_ocid_inexistente	no	sí	sí	sí
dominio_item_de_otro_proceso	no	sí	sí	sí
dominio_falta_item_en_proceso_de_104_items	no	sí	sí	sí
transitorio_una_vez	no	no	sí	sí
transitorio_siempre	no	sí	sí	sí
lento_siempre	no	no	sí	sí
salida_mal_formada	no	sí	sí	sí

- Los otros 6 casos (los 4 válidos, lento_una_vez y error_interno) los cumplen las cuatro políticas
- A cumple error_interno por coincidencia: todo error le sale como execution

Lo que B no resuelve

- B cumple 14 de 16. Los dos casos que no cumple son de infraestructura, no de entrada
- transitorio_una_vez: el índice falla en el primer intento
  - B devuelve el error transient y se rinde, aunque un segundo intento habría funcionado
- lento_siempre: la llamada tarda más de lo permitido
  - B no tiene timeout, así que espera hasta el final y devuelve ok cuando la especificación pide timeout
- Para esos dos casos hacen falta un timeout y un reintento
  - Eso es lo que agregan C y D

C y D: mismo cumplimiento, distinto gasto

Caso	Intentos en C	Intentos en D
dominio_ocid_inexistente	3	1
dominio_item_de_otro_proceso	3	1
dominio_falta_item_en_proceso_de_104_items	3	1
error_interno	3	1
salida_mal_formada	3	1

- C repite 2 veces más estos 5 casos: 10 intentos inútiles
- La latencia media pasa de 34 ms en D a 58 ms en C
- Aquí no rompe nada, porque los fallos son deterministas y la herramienta no escribe
    - Con una herramienta que escriba, o con un deadline más corto, el reintento ingenuo sí rompería
- D reintenta solo los casos con transient y timeout, que son los que pueden arreglarse

Pruebas automáticas

- tests/test_herramienta.py tiene 22 pruebas (pytest); todas pasan (results/exp3/pytest.txt)
- Cubren
  - la ejecución válida y el contrato de salida
  - 5 entradas inválidas y 3 errores de dominio sin reintento
  - transitorio una vez (se recupera), sin reintento (falla) y persistente (se corta a los 3 intentos)
  - lento una vez (timeout y luego ok) y lento siempre (respeta el deadline)
  - error interno y salida mal formada, sin reintento
  - herramienta desconocida, la función debe_reintentar, y que el contrato detecta alertas incoherentes
- Las pruebas fijan el comportamiento esperado; el experimento lo compara entre las cuatro políticas

Limitaciones

- Los fallos son simulados y deterministas: un transitorio real puede durar distinto y repetirse
    - Los tiempos solo prueban que el runtime respeta sus límites, no cómo se comporta el SEACE real
- 16 casos elegidos por nosotros: cubren cada tipo de error una o dos veces, no su frecuencia real
- El cumplimiento depende de la especificación que escribimos; otra especificación daría otro conteo
- Los límites de tiempo (100 ms, 300 ms, 3 intentos) se fijaron a mano y no se probaron otros valores
- La herramienta solo lee. Los resultados sobre reintentos no se pueden extender a una herramienta que escribe
- La latencia es de una sola corrida en una máquina

Conclusión

- Lo que más cambia el comportamiento es el contrato de entrada
  - Sin él, 3 de 3 entradas inválidas se ejecutan y una de ellas devuelve ok
  - Con él, A pasa de 6 a 14 casos cumplidos (B)
- El timeout y el reintento agregan los 2 casos que B no cubre
- El reintento selectivo (D) recupera los fallos transitorios igual que el ingenuo (C)
  - Pero sin repetir errores que no se arreglan reintentando: 19 intentos y no 29
- Una salida rota no llega al LLM: B, C y D la bloquean como output_contract
- Los fallos son simulados; los tiempos no describen al SEACE real